

Executive Productivity Agent — System Architecture

Autonomous Context Synthesis, Dynamic Drift Tracking & Strict Anti-Hallucination Architecture

Candidate: Aniruddh Mishra | **Institution:** Bennett University (B.Tech CSE) | **Target:** Arjun Malhotra (VP Sales) | **Horizon:** 21–25 Sep 2026

1. Executive Summary & Design Philosophy

The **Executive Productivity Agent** is an enterprise AI assistant designed for Arjun Malhotra, VP of Sales at Veridian Corp. Sales leaders operate under immense information fragmentation: commitments slip casually across email chains, meeting agreements evolve unrecorded, and critical administrative tasks float unassigned across departmental boundaries. Generic generative AI platforms are fundamentally unsuited for this workload because LLMs hallucinate ownership, assume authority where none exists, and fabricate status progress. To solve this, our architecture enforces a strict **Dual-Engine Hybrid Topology**:

- **Deterministic Precedence Engine:** Guarantees mathematical and logical certainty over contractual obligations, explicit deadlines, and unowned tasks (e.g. Mumbai lease renewal). Zero hallucinations permitted.
- **Grounded Semantic RAG Engine:** Employs LangChain, ChromaDB dense vector indexing, Sentence-Transformers, and Google Gemini 1.5/3.6 Flash (temperature=0.0) with strict source attribution.

2. End-to-End System Architecture

Ingestion & Normalization Layer	Intelligence & Reasoning Layer	Serving & Governance Layer
• Meeting Transcript: 4-attendee Leadership Sync dialogue extraction. • Calendars: 4 distinct schedules (Arjun, Neha, Raghav, Divya). • Email Inboxes: 5 threads × 5 messages (25 total emails). • Voice Notes: 2 personal audio memos treated as internal commitments. • Pydantic Validation: Strict schema enforcement.	• ChromaDB Vector Store: Dense embeddings (all-MiniLM-L6-v2). • Deterministic Guardrail Engine: Precedence rules for unassigned items. • Calendar Interval Calculator: Mathematical focus window protector. • LangChain Orchestrator: Google Gemini 1.5/3.6 Flash zero-shot chain.	• FastAPI REST API: Endpoints for briefing, querying, and slot search. • Streamlit Executive UI: High-contrast dashboard with live filters. • Vercel Serverless: Stateless edge deployment (api/index.py). • SQLite Audit Trail: Query, model, and timestamp logs (agent_audit.db).

3. Executive Deliverable & Commitment Matrix

ID	Deliverable	Owner	Deadline	Status Badge	Priority
COM-01	Updated Vendor List	Arjun Malhotra	Wed 23 Sep morning	AT RISK / OPEN	HIGH
COM-02	Q3 Campaign Deck Review	Arjun & Neha Kapoor	Thu 24 Sep, 9:30 AM	SCHEDULED	HIGH
COM-03	July Expense Variance Report	Divya Rao	Wed 23 Sep evening	COMPLETED	MEDIUM
COM-04	Meridian Logistics Call	Arjun & Priya Nair	Wed 23 Sep, 3:00 PM	CONFIRMED	HIGH
COM-05	Mumbai Office Lease Renewal	UNASSIGNED	Fri 25 Sep, End of Day	CRITICAL / UNOWNED	CRITICAL

4. Guardrail Logic & Evaluation Case Studies

Case Study A: Safe Ambiguity Handling (The Mumbai Lease Renewal)

The Ground Truth Scenario: Facilities sent two all-staff alerts stating an authorized signature is due by Friday 25 Sep end-of-day. In the Monday sync, Divya speculated: *'I think that's supposed to be Facilities, but I haven't seen anyone pick it up.'*

Executive Intervention: Arjun explicitly ordered: *'Okay, flag it, don't assume.'*

Corroborating Facts:

- On Wednesday 9:30 AM, Divya clarified via email that it sits with Facilities typically, but neither picked it up.
- On Thursday 4:45 PM, Raghav confirmed: *'This is now one day out and still unowned — can you confirm who's handling it?'*
- In Voice Note 1, Arjun noted: *'Someone needs to own that, I don't think it's me.'*

Engineering Decision: The agent strictly returns **UNASSIGNED / CRITICAL BLOCKER** and refuses to hallucinate Facilities as the owner. This protects leadership from assuming an urgent lease contract is managed when it is stalled.

Case Study B: Dynamic Deadline Drift (The Updated Vendor List)

Timeline of Drift:

1. *Monday 9:00 AM (Sync):* Arjun commits to sending Raghav the updated vendor list by end of day Tuesday.
2. *Monday 5:40 PM (Email):* Arjun delays: *'Running behind, will send first thing tomorrow morning instead.'*
3. *Tuesday 6:30 PM (Email):* Arjun delays again: *'Sorry, got pulled into board prep — will send by tomorrow (Wednesday) morning for sure.'*
4. *Wednesday 8:45 AM (Email):* Raghav follows up: *'Just checking — still good for this morning?'*

Engineering Decision: Because no subsequent email records the file being sent, the agent classifies status as **AT RISK / OPEN (Overdue)** and raises an immediate action alert on Arjun's briefing.

5. Deterministic Calendar Intelligence Engine

LLMs frequently miscalculate time ranges when given multiple busy blocks. Our system computes availability mathematically via interval subtraction over working hours [09:00, 18:00]:

`Available_Slots = [09:00, 18:00] \ ■ (Event_Start_i, Event_End_i)`

Protected Focus Windows on Thursday 24 Sep:

- **Board Prep Session (09:00 – 10:00 AM):** Inviolable focus block; protected from meeting booking.
- **Neha's Campaign Deck Review (09:30 AM):** Scheduled to brief Arjun ahead of Board Prep.
- **Hiring Panel (16:00 – 17:00 PM):** Blocked for interviewing sales candidates.
- **Computed Free Windows:** Correctly isolates 14 collision-free 30-minute meeting slots between 10:00 and 16:00 and between 17:00 and 18:00.

6. Governance, Observability & Cloud Deployment

- **SQLite Audit Persistence (agent_audit.db):** Every incoming question, generated answer, engine type, and timestamp is stored permanently for corporate auditing.
- **Vercel Serverless Architecture:** Configured via `vercel.json` and `api/index.py` with a lightweight package bundle (~38 MB) adhering to Vercel's 500 MB ceiling.
- **Streamlit Community Cloud & Render:** High-contrast dashboard with zero-lag responsiveness deployed from GitHub.
- **Zero Outbound Mutation Policy:** The agent drafts decisions and advises the VP of Sales, but requires explicit human confirmation before sending emails or updating Google/Outlook calendars.